

Portable Game Notation (PGN) File Format – Technical Documentation for Complex Game Trees

Overview of PGN Format Structure

Portable Game Notation (PGN) is a plain-text format for recording chess games, designed to be both human-readable and machine-readable. A PGN file typically contains one or more chess games. Each game is divided into two main sections: a **Tag Pair Section** (metadata) and a **Movetext Section** (the moves of the game). The movetext section concludes with a **game termination marker** indicating the result of the game ¹ ². In essence:

- **Tag Pair Section:** A series of metadata tag pairs (like Event, Date, Player names, etc.) enclosed in brackets `[...]` at the beginning of the game.
- **Movetext Section:** The sequence of moves in Standard Algebraic Notation (SAN), along with any annotations (comments, NAGs, variations), ending with a result marker (e.g. `1-0`, `0-1`, `1/2-1/2`, or `*`).

Multiple games in a single PGN file are separated by one or more blank lines. PGN defines two formatting standards: - **Import format:** a flexible format intended for data that may be entered or edited by hand (parsers should be lenient with spacing, line breaks, etc.). - **Export format:** a stricter format for consistent program output (e.g. one tag pair per line, 80-character line width for moves, etc.), suitable for archival storage ³ ⁴.

PGN data is plain text (ASCII/Latin-1 by standard) for broad compatibility ⁵. Unicode characters are generally not expected in the movetext; piece names and move notation use English letters. (Player names and comments may include non-ASCII characters if the encoding supports it, but software should degrade gracefully (e.g., replace unsupported characters with `?`) for maximum portability ⁶.)

Game Termination Marker: Every movetext section ends with a result code matching the game's Result tag. This marker is always one of four values: `"1-0"` (White wins), `"0-1"` (Black wins), `"1/2-1/2"` (drawn game), or `"*"` (game in progress or result unknown) ² ⁷. For example, a game still ongoing or without a recorded result is indicated by `Result "*"`, and the movetext would end in `*` ². (Note: these use the digit zero `0`, not the letter `O`, which helps distinguish results from notation like castling ⁸.)

Below is a high-level example of a simple PGN game structure:

```
[Event "F/S Return Match"]
[Site "Belgrade, Serbia JUG"]
[Date "1992.11.04"]
[Round "29"]
[White "Fischer, Robert J."]
[Black "Spassky, Boris V."]
```

```
[Result "1/2-1/2"]
```

```
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. O-O Be7 6. Re1 ... 1/2-1/2
```

In this example, the tag section provides game metadata (Event, Site, Date, etc.), and the movetext section lists the moves (1. e4 e5 2. Nf3 Nc6 ...) concluding with the result 1/2-1/2 (a draw).

PGN Tag Pair Section (Metadata Tags)

The **tag pair section** contains metadata that identifies and describes the game. It consists of one or more **tag pairs**, each on its own line in export format (import format may allow multiple on a line). A **tag pair** is always written as a left bracket [followed by a case-sensitive tag name (a symbol token), then a string value in quotes, and a right bracket]⁹. For example:

```
[White "Alice"]
```

Here "White" is the tag name and "Alice" is the tag value. Tag names must be composed only of letters, digits, and underscores (no spaces or special characters)⁵. Tag names should not repeat within the same game's tag section¹⁰. In PGN export format, each tag pair should appear on its own line, and an empty line follows the last tag to separate it from the movetext¹¹.

Seven Tag Roster (STR): PGN defines seven *mandatory* tags, known as the *Standard Seven Tag Roster*, which serve as the minimal information to identify a game. These seven tags, which should appear first (and in this exact order in export format), are¹²:

1. **Event** – Name of the tournament or event (e.g. "FIDE World Championship"). Use "?" if unknown¹³.
2. **Site** – Location of the event. Typically "City, Region COUNTRY", where country is an IOC country code (e.g. "New York, NY USA"). Use "?" if unknown¹⁴¹⁵.
3. **Date** – Start date of the game in YYYY.MM.DD format. Use "?????.??.??" for unknown parts¹⁶¹⁷.
4. **Round** – The round number (or game number in a match). Use "-" if not applicable or "?" if unknown¹⁸¹⁹.
5. **White** – The player of the white pieces (last name, first name format, or other identifier)²⁰.
6. **Black** – The player of the black pieces (same format as White)²¹.
7. **Result** – The game's result, identical to the game termination marker at movetext end⁷. Must be one of "1-0", "0-1", "1/2-1/2", or "*".

These tags provide essential context. For example, a minimal PGN for an unfinished game might include something like:

```
[Event "Casual Game"]  
[Site "Berlin GER"]  
[Date "2025.08.20"]  
[Round "1"]  
[White "Alice"]
```

```
[Black "Bob"]  
[Result "*"]
```

In import format, tag pairs can appear in any order (and even span multiple lines if needed), but in export format they should be ordered as above for the STR, followed by any additional tags sorted alphabetically by tag name ²². A blank line then signifies the end of the tag section ²³ ²⁴.

Additional Tags: Beyond the seven standard tags, PGN allows any number of additional, optional tag pairs to record further information. Many conventions exist, and the PGN specification itself suggests numerous supplementary tags (section 9 of the standard). Commonly used optional tags include:

- **Player title and ratings:** `WhiteTitle`, `BlackTitle` (e.g. "GM", "IM") ²⁵; `WhiteElo`, `BlackElo` (numeric Elo ratings) ²⁶; `WhiteUSCF`, `BlackUSCF` (if relevant national ratings) ²⁷.
- **Event details:** `EventDate` (start date of the event) ²⁸, `EventSponsor`, `Section`, `Stage`, `Board` (board number in team events) ²⁹ ³⁰.
- **Opening information:** `ECO` (Encyclopedia of Chess Openings code, e.g. "C65") ³¹, `Opening`, `Variation`, `SubVariation` (human-readable opening name and variation) ³² ³³.
- **Miscellaneous:** `Annotator` (who annotated the game), `TimeControl` (e.g. "40/120:5/60:Sudden Death/30"), `Mode` (e.g. "OTB" or "ICS" for over-the-board or online), `Termination` (e.g. "Normal" or "Abandoned" – reason game ended) etc., which some databases use ³⁴.

There are also tags for non-standard starting positions: - **SetUp** – if present and set to "1", indicates the game did not start from the standard initial chess position ³⁵. - **FEN** – gives a FEN string for the starting position if `SetUp "1"` is used ³⁶.

By including `SetUp` and `FEN` tags, PGN can represent positions from chess variants or partial positions (common in problem setups or Fischer Random/Chess960 initial positions). For example:

```
[SetUp "1"]  
[FEN "rnbqkbnr/pppppppp/8/8/3P4/8/PPP1PPPP/RNBQKBNR b KQkq - 0 1"]
```

would indicate the game starts from a position where a pawn has been moved to d4, and it's Black to move.

Formatting notes: In export format, there should be no whitespace between the `[` and tag name, or between tag name and the quoted value, etc. (each tag pair is one line, neatly formatted) ³. In import format, tags can be looser (even multiple on one line, though that is rare). Tag values are quoted strings and cannot contain newline characters or unescaped quotes; the maximum length for a tag value is 255 characters ³⁷. If a tag value itself needs to include a quote character, it can be represented by `\` within the string.

Movetext Section (Moves and Annotations)

After the tag section's blank line, the **movetext section** describes the moves of the game, along with any annotations (comments, variations, etc.). The movetext is comprised of a sequence of move notations (in SAN), potentially interspersed with **move numbers**, **punctuation or NAGs** for evaluations, **comments**, and **variations**, and it **ends with the result code** ³⁸.

Standard Algebraic Notation (SAN): Moves are given in standard algebraic chess notation. This uses uppercase letters for piece types (K, Q, R, B, N for king, queen, rook, bishop, knight; pawns are not given a letter), followed by the destination square (e.g. `e4`, `Nf3`). Disambiguation (if two pieces of the same type can move to the same square) is handled by adding either the file or rank (or both) of the originating square as needed (e.g. `Nbd7` means the knight from the b-file goes to d7). Captures are indicated by an `x` (e.g. `Qxb7` means the queen captures something on b7). Pawn moves that capture include the file of origin, e.g. `exd5` (pawn from e-file takes something on d5). Special notations include `0-0` for kingside castling and `0-0-0` for queenside castling (using the letter "O", not zero)³⁹, and e.g. `e8=Q` to indicate pawn promotion (pawn moves to e8 and becomes a Queen). A plus `+` or hash `#` for check or checkmate can appear in SAN in published games, but PGN typically does not require these symbols and they are often omitted or handled as annotations, not as part of the movetext token stream.

Each **full move** in SAN consists of White's move followed by Black's move (if any) for that move number. In PGN export format, move numbers are included for clarity. For example:

```
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6
```

indicates White's 1st move was e4, Black's 1st was e5; White's 2nd was Nf3, Black's 2nd Nc6; White's 3rd Bb5, Black's 3rd a6. The move number with a period (like `3.`) precedes White's move, and Black's move is written after White's on the same line. If a black move is shown without its preceding white move (e.g., when a game starts with Black, or a variation starts from Black's turn), it is prefixed with an ellipsis, e.g. `1... Nc6`⁴⁰. PGN's export conventions say that a black move number indication uses three dots after the number (e.g. `5...`), whereas a white move uses one dot `5.`⁴⁰. In import format, PGN parsers are usually tolerant of missing or repeated move numbers, as long as the sequence is clear and correct⁴¹.

Game Moves and Result: Moves continue until the game's end. The movetext section *must* terminate with the game termination marker (the result). This result token (e.g. `1-0`) is written as a standalone token at the end, not as a move. It confirms the outcome and should match the `[Result]` tag in the header². For instance, if the Result tag is `"0-1"`, the movetext must conclude with `0-1` as the last token, after Black's final move. PGN does not allow moves after the game is decided; any post-game commentary must come before the result marker. If the result is `*` (unknown or ongoing), `*` appears at movetext end to indicate an uncompleted game.

Illegal moves: By definition, PGN movetext should not contain illegal chess moves. All moves are expected to be legal from the given position (assuming standard rules). Illegal moves, if they appear at all, would only be in comments/annotations (for example, a commentator might mention a hypothetical illegal move in a comment for instructional purposes)⁴². PGN readers are generally expected to enforce legality or at least not crash on illegal moves, but PGN files with illegal moves are non-conformant unless those moves are quarantined in comments.

Example Movetext (simple):

```
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. 0-0 *
```

This indicates a game where after 5. O-O (White castles king side), the game is still ongoing or stopped (result `*`). If White had won instead, one would see `1-0` at the end and the Result tag would be `"1-0"`.

Comments and Commentary in PGN

PGN allows **comments** (annotations in natural language or evaluation symbols) to be embedded in the movetext. Comments are not moves but notes, and a PGN parser should ignore them when extracting the moves. There are two types of comment syntax defined ⁴³:

- **Semicolon comments:** A semicolon `;` introduces a comment that continues to the end of the line. This is like a “rest-of-line” comment. For example:

```
1. e4 e5 2. Nf3 Nc6; Black develops a knight
```

Everything after the `;` on that line is a comment (here, "Black develops a knight").

- **Brace comments:** A pair of curly braces `{ ... }` encloses a comment. This comment may span spaces and even line breaks (in import format) until the closing brace. For example:

```
2... Nc6 {Black develops a knight toward the center}
```

Brace comments can span multiple lines if needed (import format), but they cannot nest – the first `}` encountered will close the comment ⁴⁴. Inside a comment, braces or semicolons lose their special meaning (they are treated as literal characters) ⁴⁴, which means you **cannot** start a new comment inside an existing one.

Comments **cannot appear in the middle of a token** ⁴⁵. This means you must place comments between moves or after an entire move notation, not in the middle of a move notation. For example, `Nf{comment}3` is invalid because it breaks the move token "Nf3". Instead, place the comment either before or after the move token with a space separating it.

In practice, brace-enclosed comments are far more common than semicolon comments in PGN files, because they can be placed inline between moves. Semicolon comments are typically used at end of lines. A PGN parser can treat both types similarly (e.g. skip everything from `;` to end of line, or from `{` to `}` as needed).

Placement of comments: Comments may appear: - **Before a move** (on its own or between move number and move, though this is less common). - **Immediately after a move** (this is typical for post-move commentary). - **Before a variation** (i.e., a comment right before an `(` indicating a variation, often to explain the upcoming variation).

For example:

```
5. O-O {White castles; this is a crucial positional move} 5... Be7  
  ( {Instead of 5...Be7, Black can try an alternative:} 5... b5 6. Bb3 ... )
```

In this snippet, there's a comment after White's move 5. O-O, and a comment immediately after the parenthesis that begins the variation on Black's 5th move. The latter is a **pre-variation comment** – it describes the variation that follows.

PGN doesn't formally distinguish "post-move" vs "pre-move" comments – all comments are just whitespace to the parser. However, by convention: - A comment directly following a move (on the same line or separated by space) is taken as a comment on that move. - A comment appearing right before a variation (`(`) is understood as an introduction to that variation. - A comment not attached to a specific move (e.g., at the start of the movetext, or between the result and a move) might be a general comment on the position.

Comments with special content: Many chess programs embed additional data in comments. For instance: - Clock times for each move are often recorded in braces, e.g., `1. e4 {[%clk 1:40:30]}` `1... c6 {[%clk 1:40:25]}` might indicate remaining clock after each move. Some formats simply put the time in braces like `{1:40:30}` without a `[%clk]` tag, relying on convention. - Engine evaluation annotations can also appear in comments, often with a prefix like `[%eval 0.34]` or similar.

These `[%...]` notations are **not official PGN standard**, but are widely used in practice (for example, Lichess "extended PGN" includes evaluation and clock info in this manner). Since they are inside `{}` braces, they remain valid PGN (just as comments). Your application can choose to parse them specially if needed or ignore them as generic comments. The key is that such content is encapsulated in comments so that programs not aware of them will simply skip them.

Variations and Recursive Annotation Variations (RAV)

One of PGN's powerful features is the ability to incorporate **variations**, i.e., alternate lines of moves representing "what-if" scenarios or analysis lines. In PGN, a variation is written as a sequence of moves enclosed in parentheses (`(`)). Variations can be nested within one another, leading to a branching **game tree** of moves – hence the term *recursive annotation variation* (RAV) ⁴⁶.

Basic variation syntax: An RAV is "an alternate move sequence that may be legally played by first *unplaying* the move that appears immediately before the RAV" ⁴⁶. In practice, this means: - You write a parenthesized list of moves *right after* the move where a branching occurs. - When reading the game, to follow the variation, you conceptually undo the preceding move (the point of divergence) and then play the moves in the parenthesis.

Example:

```
8. h4 (8. h3?! an alternative slower plan) 8...Nc6
```

Here, after White's move `8. h4` in the main line, a variation is given: `(8. h3?! ...)`. This indicates that instead of `8. h4` (the main move), White could have played `8. h3`. The variation line might then show Black's response and so on, until the closing `)`. After the variation is closed, the main line continues with Black's actual move `8...Nc6`. The mainline move that immediately precedes a variation is the move from which the variation branches.

Move numbering in variations: Notice in the above example, the variation started with a White move at move 8, so it repeated the move number `8.` inside the parentheses. If a variation begins with a black move (i.e., an alternative for Black's response), it should include the ellipsis. For example:

```
5... Be7 (5... b5 6. Bb3 ...) 6. Re1 ...
```

In this case, Black's 5th move in the main line was `5... Be7`. The variation given is an alternative black 5th move `(5... b5 6. Bb3 ...)`. The `5...` with ellipsis inside the parentheses indicates "instead of `5...Be7`, Black could play `5...b5`." Within that variation, after `5... b5`, White responded with `6. Bb3` (the variation continues with White's 6th, etc.). If multiple moves for White and Black are inside the variation, the numbering proceeds accordingly. PGN parsers often ignore move numbers inside variations as redundant, but including them (especially at the start) makes the PGN more readable for humans and is standard in well-formatted PGNs.

Multiple alternatives at one point: PGN supports listing more than one alternative move at the same juncture. This is done by sequentially providing multiple parenthesized sequences one after another, all following the move from which they branch. For example:

```
4. Bb5 a6 5. Ba4 (5. Bxc6+?! one alternative) (5. Be2 another option) 5...  
Nf6
```

Here, after White's `5.Ba4` in the main line, two separate variations are given: one for `5. Bxc6+?!` and one for `5. Be2`. These are **parallel variations** (siblings) branching from the same position (after Black's `4...a6` and before White's 5th move). Each variation is enclosed in its own parentheses. They are read as independent alternatives. The main line then continues with Black's actual `5...Nf6` after listing the alternatives.

Nested variations (sub-variations): Inside a variation, you can further embed another variation to show alternatives at a deeper point. This creates a nested parenthesis structure. For instance:

```
5... Be7 ( {Instead, Black can try:} 5... b5  
6. Bb3  
(6... d6 7. d4 exd4 8. Nxd4)  
6... Na5 7. Nxe5 ...) 6. Re1 ...
```

In this example, the main line has `5... Be7`. Then one variation is given for Black's 5th move: `5... b5 6. Bb3 ...`. Within that variation, there is a **nested** variation after `6. Bb3` showing another line: `(6... d6 7. d4 exd4 8. Nxd4)`. This nested variation is a sub-variation exploring what if Black played `6... d6` instead of the `6... Na5` that appears right after it in the parent variation. Once the inner parenthesis closes, the outer variation resumes (with `6... Na5 7. Nxe5 ...` in this case), and eventually closes, returning to the main line at move 6.

In summary, parentheses in PGN form a recursive structure: - An open parenthesis `(` starts a variation branching off the current move. - A closing parenthesis `)` ends the current variation and returns to the *one level higher* context (which could be the main line or an outer variation). - Variations can be nested to any depth, though in practice deeply nested lines (more than 2-3 levels) are uncommon and some software might not handle extremely deep nesting ⁴⁷. PGN itself places no fixed limit on nesting.

To interpret nested variations: *"the nested structure is composed of independent variations; only if there's a closing parenthesis does it end that branch. For example, `8... Qc7` might be a main variation move, `(9.Qe1 ... )` a subvariation, and `(10.Bf3 ... )` a sub-subvariation. The nesting looks like:*

(8... Qc7 (9.Qe1 ... (10.Bf3 ...))).”⁴⁸ In other words, each opening parenthesis starts a new branch; a closing parenthesis ends the latest open branch.

RAV in PGN vs software support: It’s worth noting that not all chess software fully supports reading and displaying recursive variations beyond a certain depth. PGN is capable of representing them (and they are considered valid PGN), but some GUI viewers only show one level of alternatives or treat deeper ones as comments. For instance, older software like Palview3 was limited to two levels of variations in output (three levels counting the main moves)⁴⁷. Modern PGN readers usually handle multiple levels, but if you plan on deep nesting, ensure to test with your target tools.

Numeric Annotation Glyphs (NAGs)

Numeric Annotation Glyphs are a mechanism in PGN to provide standardized symbolic evaluations of moves or positions in a language-independent way⁴⁹. In traditional chess notation, you might see symbols like **!** (good move), **?** (bad move), **!?** (interesting but possibly dubious move), etc. NAGs generalize these and many more by using a numeric code. A NAG is represented as a dollar sign **\$** followed by a number from 0 to 255⁴⁹. For example, **\$1** corresponds to **!** and **\$2** corresponds to **?**.

Syntax: In the movetext, a NAG is a distinct token, e.g., **\$3** would appear separated by a space from the move it annotates. Formally: “A NAG is a token composed of a dollar sign **\$** immediately followed by one or more digits (0-9). It is terminated by the first non-digit character after the digits.”⁴⁹ So **\$15** and **\$5** are two different NAGs (meaning NAG number 15 and 5 respectively).

Standard meanings: The PGN specification defines an interpretation for many NAG numbers⁵⁰. Some commonly used NAGs include:

- **\$1** – Good move (equivalent to **!**)⁵¹
- **\$2** – Poor move (**?**)⁵¹
- **\$3** – Very good move (**!!**)⁵²
- **\$4** – Very poor move (**??**)⁵³
- **\$5** – Speculative or interesting move (**!?**)⁵⁴
- **\$6** – Questionable move (**?!**)⁵⁵

Those cover the common punctuation. But NAGs go beyond **!** and **?**. For example: - **\$7** – Forced move (all others lose quickly)⁵⁶

- **\$10** – Drawish position (evaluation symbol indicating likely draw)⁵⁷

- **\$14** – White has a slight advantage⁵⁸ (similarly \$15 for Black slight advantage, \$16 White moderate, \$17 Black moderate, etc.)

- **\$18** – White has a decisive advantage (and \$19 for Black decisive advantage)⁵⁹

- There are NAGs for various specific positional comments like \$22 = White in zugzwang, \$32 = White has a moderate time advantage, \$132 = White has moderate counterplay, etc. A full table is in the spec^{60 61}.

The spec categorizes them as: - 0 is reserved (null annotation)⁶². - NAGs 1–9 apply to the move just made (usually punctuation and immediate move quality)⁶³. - NAGs 10–135 describe evaluations of the resulting position (positional assessments like “unclear position”, “White has a slight advantage”, etc.)⁶³.

- NAGs 136–139 relate to time pressure (e.g., \$136 means White has moderate time trouble)⁶⁴. - The rest are reserved for future or other uses⁶⁵.

Usage: In PGN export format, it's recommended to use NAGs rather than literal `!` or `?` characters in movetext, to avoid encoding issues and keep annotations language-neutral ⁶⁶. For example, instead of writing `Qxa8?`, one would write `Qxa8 $2` in a strict PGN. The symbol `?` is then inferred by any software that knows `$2` means a mistake ⁶⁶. In practice, many PGN files (especially those entered by hand or exported by some software) might still contain literal `!` and `?` attached to moves. PGN import should be able to handle these in many cases. For instance, some parsers treat a move like `Qxa8?` as equivalent to `Qxa8 $2`. But when generating PGN, using NAGs is the standard way to include these annotations.

Multiple NAGs can follow a move (though it's rare). They should be space-separated tokens after the move. For instance, `Bg5 $1 $18` would mean the move Bg5 is a good move (`$1`, `!`) and that it gives White a decisive advantage (`$18`).

Example:

```
13. Bxf6 $3 $36 {White sacrifices the bishop for a strong attack}
```

Here `$3` (very good move, `!!`) and `$36` (White has the initiative) are both applied to 13.Bxf6, and a comment follows explaining it.

Extended PGN (PGN+): Variations and Annotations in Practice

The term "PGN+" or *extended PGN* is informally used to refer to PGN data that goes beyond simple move recording – including extensive annotations, multiple variations, engine analysis, time stamps, etc. Standard PGN (as per the 1994 spec) supports much of this via RAV and NAG and comments, but not all chess software supports or uses all features. Here's how extended features are handled within PGN:

- **Nested Game Trees:** As described, PGN can represent complex game trees with recursive variations. Use this to include deep analysis lines. Keep in mind some consumers might only display the main line or a limited number of branches ⁴⁷.
- **Inline Commentary:** Use `{ ... }` comments liberally to convey textual annotations, explanations, or even structured data (like time/eval). This is the officially supported way to embed any notes in PGN.
- **Language Independence:** Use NAGs for common symbols instead of literal `!` `?` to remain language-neutral. For longer textual evaluations, you may still include them in comments (e.g. `{White stands better}` or `{with a winning advantage}`), which is fine – just remember these comments won't be understood by machines beyond being skipped.
- **Variations vs. Comments:** PGN's design encourages using **variations** to show alternative move sequences rather than describing them in prose. Instead of writing in a comment "Better is 12.Nf5", it's more structured to actually include the line `(12. Nf5! ... )` as a variation. Comments can still accompany variations to explain *why* one move is better.

Example of a Complex PGN with deep variations and comments: Below we present a snippet of a PGN game that illustrates many features together – tag pairs, mainline moves, a branched game tree with multiple and nested variations, comments, and NAGs. This example is for illustration purposes (not a real game), demonstrating how a richly annotated PGN might look:

```

[Event "Analysis Example"]
[Site "Exampleville"]
[Date "2025.08.20"]
[Round "1"]
[White "Alice"]
[Black "Bob"]
[Result "*"]
[WhiteElo "1500"]
[BlackElo "1500"]
[Annotator "Alice"]

1. e4 {White opens with the King's pawn} e5 {Black responds symmetrically}
2. Nf3 Nc6
3. Bb5 a6
4. Ba4 Nf6
5. O-O Be7
  ( {Instead, Black can try the aggressive Archangel line:}
    5... b5 6. Bb3
      ( {Black could also play 6... d6 here to defend e5}
        6... d6 7. d4 exd4 8. Nxd4 $2
          {This pawn grab is considered dubious in modern theory}
        )
      6... Na5 7. Nxe5 d6
    )
6. Re1 b5
7. Bb3 O-O
8. h3 {Preventing ...Bg4} (8. a4 $5 {An interesting pawn sacrifice} b4 9. d4
  $1 {White seizes the center} 9... exd4 10. e5 * )
9. d4 exd4
10. e5 $1 {A powerful thrust!} Ne8
11. Nxd4 $6 {Perhaps too materialistic; better was 11.Nc3} Nxd4
12. Qxd4 c5
13. Qe4 Rb8
14. Nc3 *

```

Let's break down what's shown:

- Tag section includes standard tags and some optional ones like `WhiteElo`, `BlackElo`, `Annotator`.
- Moves are numbered and in SAN (e.g. `O-O` for castling, `exd4` for pawn capture).
- Comments are included in `{ }` after moves (e.g. after 1. e4, 1...e5, 8. h3, 10. e5, 11. Nxd4).
- A major variation is given after Black's 5th move: `(5... b5 6. Bb3 ... 7. Nxe5 d6)`, introduced by a comment. Inside that variation, a sub-variation is nested for Black's 6th move: `(6... d6 7. d4 exd4 8. Nxd4 $2 {...})`.
- NAGs: `$2` after 8. Nxd4 indicates it's considered a poor move (`( ? )`), and the comment explains why. `$5` after 8. a4 marks it as an interesting idea (`( !? )`), `$1` after 9. d4 marks a good move (`( ! )`), `$6` after 11. Nxd4 marks a questionable move (`( ?! )`).
- Another variation after 8.h3: `(8. a4 $5 ... 10. e5 * )` showing an alternate sequence where we ended it with `*` because it's an analysis line not played to an actual result.

- The game termination marker in this example is `*` (since we marked Result as "*" for an ongoing/analysis game). If this were a real game, the moves would continue until a 1-0, 0-1 or 1/2-1/2 is reached and that would replace the final `*`.

This example demonstrates how complex a PGN can get, with deep variations and multiple annotations. It is **fully valid PGN** (PGN can handle it), though not every chess program may display it elegantly due to the nested variations.

Generating and Parsing PGN: Practical Considerations

When building applications to read/write PGN, you need to handle the above structures carefully. Here are key considerations and some strategies for parsing and serializing:

Parsing PGN (Reading)

1. **Split Header and Moves:** First, separate the tag pair section from the movetext. For example, you can read the file until you hit an empty line; everything before that is tags, everything after is moves.
2. **Parse Tag Pairs:** Each tag pair is of the form `[TagName "TagValue"]`. A simple approach is to use a regular expression or manual parsing to extract the tag name and value. For instance, a regex like `^(`

`(\w+)\s+"(["]*)"`

can capture tag name and value (assuming one per line, which is true in export format). Make sure to accumulate tag pairs into a dictionary or map for the game. Handle escape sequences if any (though PGN doesn't have many special escapes except `\` inside values).

3. **Tokenize Movetext:** The movetext can be tokenized into meaningful units:
4. **Move number indicators** (e.g. `1.` or `1...`) – these can essentially be ignored for the purposes of building the game tree, but you might use them to verify correctness or to format output.
5. **SAN move tokens** (like `e4`, `Nf3`, `O-O`, `exd5`, `Qa1+`, etc.).
6. **Game termination marker** (`1-0`, `0-1`, `1/2-1/2`, `*`) – marks end of moves.
7. **NAG tokens** (`$` followed by number).
8. **Comment tokens** (`{` to `}` and `;` to end of line).
9. **Variation tokens:** `(` and `)`.

A robust approach is to iterate through characters of the movetext and build tokens: - When you see a digit followed by a period, that's a move number (possibly consume the `...` if present). - When you see a `$`, consume digits after it as one token (NAG). - When you see `{`, consume everything until the matching `}` as one comment token (skipping any braces inside as plain text). - When you see `;`, consume everything until the end-of-line as a comment token. - When you see `(` or `)`, those are separate tokens. - Everything else (letters, numbers, punctuation like `+`, `#`, `=`) that forms a

contiguous string without spaces or special characters would form a move token (or part of it). Typically SAN moves may include `=Q` or `+` etc as part of the token.

1. **Build the Game Tree:** Use a stack or recursive structure to handle variations. For example:
2. Maintain a pointer to the current move list (starting with the main line).
3. Iterate through the tokens:
 - If the token is `(`, this means a new variation begins **at the current position** (branch off the last move before this parentheses). You should push the current position onto a stack, and start a new variation line. Often one creates a new list of moves, attach it as a variation to the last move in the current line, and then descend into this new list.
 - If the token is `)`, a variation ends. You pop the last stored position off the stack, resuming the previous line.
 - If the token is a move (SAN string like `Nf3`, `O-O`, etc.), create a new move node in the current line:
 - You may need to interpret SAN to know which piece and squares, but for just storing the PGN structure, you might keep it as raw SAN or parse it later with chess logic.
 - Optionally track the move number from the last seen move number token (though this is not strictly necessary to build the tree).
 - Append the new move to the current line's move list.
 - If the previous token was a comment token, you might associate that comment with this move or with the position before this move depending on placement.
 - If the token is a NAG (like `$5`), attach it to the most recent move (e.g. add to that move's list of NAGs).
 - If the token is a comment, you can attach it to the most appropriate element:
 - If it appears right after a move token (before another move or before a variation), treat it as commentary on that move.
 - If it appears immediately after an `(` (i.e., the first token in a variation is a comment), you may want to attach it as a pre-variation comment (perhaps to the variation node or to the position).
 - If it's before any move (at start of movetext), it could be a general comment on the initial position or the game as a whole.
 - It's up to your design whether you attach comments to moves or to positions between moves. A common simple approach: attach a comment to the move that came immediately before it (or to a dummy start move if it's at the very beginning).
4. **Use Chess Logic if Needed:** If you need to validate moves or get additional info (like generating figurine notation, or checking legality), you will need a chess engine or rules implementation. PGN itself doesn't encode which piece moved explicitly (except through SAN disambiguation), so determining the from-square of a move can require knowing the current board position. Many PGN readers integrate a chess library to update the board as moves are parsed, mainly to validate SAN and handle disambiguation. For example, SAN `Nc3` must be interpreted in context if both knights can go to c3. For pure storage or translation, you might skip this, but for correctness you eventually need it.

Serializing PGN (Writing)

When outputting PGN from an internal game representation, aim to produce a standards-compliant export format for maximum interoperability. Key steps:

1. **Output Tag Section:** Write the seven standard tags first (Event, Site, Date, Round, White, Black, Result) in that order, each as `[Tag "Value"]`. Follow with any additional tags (e.g. ratings, ECO, etc.) sorted alphabetically by tag name ²². Ensure values are properly quoted and any internal quotes escaped.
2. **Blank line:** After tags, output a blank line.
3. **Output Moves and Variations:** This is essentially the inverse of the parsing process. You likely have your game stored as a structure (e.g., a main line list of moves, where each move node may contain a list of variations branching from that move).

A recursive approach works well: - Write out move numbers and moves in sequence. For each move: - If the move is a White move at the start of a full move, prepend the move number and a dot (e.g. `12.`). If it's a Black move and you are placing it on a new line (or if you want to always include it), you can prepend the move number and three dots (`12...`), but since standard formatting puts white and black moves on the same line, you often only explicitly write the number for White's move and then the black move follows. - Write the SAN of the move. - If the move has NAGs, output each as `$number` separated by space. - If the move has a comment attached, output a space then `{ comment text }`. - **Immediately after writing a move (and its trailing annotations)**, if there are one or more variation lines branching from *this move*, handle them: - For each variation, output a space then `(` to begin the variation. - Inside the parentheses, recursively output the sequence of moves for that variation *starting from the position after this move*. **Important:** Do NOT repeat the move that was just played in main line; the variation moves begin from the next side to play. However, you **should include an appropriate move number** for the first move in the variation. - If the variation starts with the same side to move as the main line had after the branching move, you repeat the move number. In the above parsing example, main line had `5... Be7` then a variation started for Black's 5th: we wrote `5... b5` inside variation. - If the variation starts with a new full move (i.e., the main line move was White's and now we are showing Black's alternative response, or vice versa), you may need to increment or use ellipses accordingly. - Output the variation moves by the same procedure (including any sub-variations recursively). - When the variation line is done (it will end before the main line's move that follows the variation), output a `)` to close the variation. - If there are multiple variations at this point, they will each be enclosed in their own parentheses one after another (with a space separating them). - Continue to the next move in the main line. - Ensure that after the last move, you output the game termination marker (result). Typically, after White's move if Black has no move (e.g., game ended on White's move or Black resigns), you still output the result marker.

Controlling line lengths and spacing: The PGN standard suggests limiting lines to 80 characters in export format ⁴. You can insert line breaks appropriately, usually after a full move or after a variation to avoid extremely long lines. Ensure you don't break a line in the middle of a move token or inside a comment or variation unexpectedly. Also, no trailing spaces at line ends, and no leading spaces at line starts except the space that naturally separates move numbers, moves, and commentary (export format says no line should start or end with a space character ⁶⁷).

Example – Code Snippet (C# Pseudocode): Below is a simplified illustration of how one might traverse a game tree to serialize moves in PGN (this assumes you have a data structure for moves). This is not a complete implementation, but outlines the approach:

```
class MoveNode {
    public string SAN;                // e.g. "e4", "Nf3", "O-O"
    public string Comment;           // comment after the move (if any)
    public List<int> NAGs = new List<int>(); // list of NAG codes for this
move
    public List<List<MoveNode>> Variations = new List<List<MoveNode>>();
    public MoveNode Next;           // next move in the main line (null if
end of line)
}

// Recursive function to print moves from a given start move
void PrintMoves(MoveNode move, int moveNumber, bool isWhiteMove) {
    MoveNode current = move;
    while (current != null) {
        if (isWhiteMove) {
            // Print move number and dot for White's move
            output($"{moveNumber}. ");
        } else if (moveNumber > 0) {
            // Print move number with ellipsis for Black's move if at line
start
            output($"{moveNumber}... ");
        }
        output(current.SAN);
        // Print NAGs if any
        foreach (int nag in current.NAGs) {
            output($" ${nag}");
        }
        // Print comment if any
        if (!string.IsNullOrEmpty(current.Comment)) {
            output($" {{{current.Comment}}});");
        }
        // Now, variations branching from this move:
        foreach (var variationLine in current.Variations) {
            output(" (");
            // variationLine is a list of MoveNode representing the variation
            // The first move in variationLine is played instead of
current.Next in main line
            // Determine numbering for variation's first move:
            int varMoveNumber = isWhiteMove ? moveNumber : moveNumber;
            bool varIsWhite = isWhiteMove;
            // If main line had a white move and we are giving a variation
for black's reply,
            // then variation starts with a black move -> use ellipsis.
            // If main line had a black move, and variation is an alternate
white move next moveNumber+1.
            if (isWhiteMove) {
```

```

        // current was white's move, so variation is an alternate
Black move on moveNumber
        varIsWhite = false;
    } else {
        // current was black's move, so variation starts at next move
number for White
        varIsWhite = true;
        varMoveNumber = moveNumber + 1;
    }
    PrintMoves(variationLine.First(), varMoveNumber, varIsWhite);
    output(")");
}
// Move to next move in main line
isWhiteMove = !isWhiteMove;
if (!isWhiteMove) {
    // If we just printed White's move, we will print Black's on same
move number
    // else if we just printed Black's, increment move number for
next White
    // (we increment when toggling from Black to White)
    moveNumber++;
}
current = current.Next;
if (current != null) output(" "); // space before next move
}
}

```

(This pseudocode assumes an output function that writes to the PGN string/stream. It toggles `isWhiteMove` as it iterates. It handles variations by recursion. In practice, you'll also need to handle outputting the final result after the loop.)

The above illustrates that writing PGN is essentially a depth-first traversal of the move tree, with careful insertion of parentheses for variations and move numbers at appropriate times. The details can be tricky, especially with move numbers in variations – ensure that the first move in a variation is annotated with the correct move number and dots (`.` or `...`) based on whose turn it is and whether the variation starts at the beginning of a turn or mid-turn.

1. **Output Result:** After printing all moves (and variations), print the result code (e.g., `"1-0"` or `"0-1"` or `"1/2-1/2"` or `"*"`). In the code above, you would handle that outside the `PrintMoves` function, once the main line traversal is done (and after any trailing space/newline adjustments).
2. **Validate Consistency:** When generating PGN, double-check that:
 3. The `[Result]` tag matches the termination marker you output ⁶⁸.
 4. If you included `SetUp` and `FEN` tags (non-standard start), the first moves make sense from that position.
 5. All tags are properly closed with `]` and all braces in comments and parentheses in variations are balanced.

Validation and Interoperability

Validation: A “valid” PGN can have varying strictness levels. At minimum: - Syntactic validity (tags properly formatted, moves and annotations properly formatted, no unbalanced parentheses or braces, etc.). - Semantic validity: moves should be legal and consistent (this you may or may not enforce in a PGN utility, but it’s good to check). - The result should reflect the final position if known (for instance, don’t mark `1-0` if Black actually delivered mate, unless that’s to indicate White resigned which is fine but the context should match).

There are utilities and libraries to validate PGN files. Using a chess engine or library to play through the moves is a common approach to ensure there are no illegal moves or ambiguous SAN that wasn’t resolved.

Common Pitfalls & Best Practices:

- **Unmatched Brackets/Braces:** Make sure every `(` has a matching `)` and every `{` has a matching `}`. A single missing parenthesis can corrupt the entire parsing of variations. Similarly, stray braces will confuse comment parsing. When editing PGN by hand or generating, always keep a stack or counter to ensure balance.
- **Over-nesting:** Deeply nested variations are valid but be aware of software limits. Some GUI viewers might ignore or flatten variations beyond a certain depth ⁴⁷. If you control the output (for example, for a publication), consider limiting depth or splitting extremely complex analysis into separate annotated games for clarity. However, chess database programs (like ChessBase, SCID, etc.) generally handle multiple levels.
- **Null moves and special notations:** PGN does **not** have a standard notation for a “null move” (a pass). In some problem contexts or engine analysis you might see `--` or `...` used to denote a pass; these are not standard and would confuse PGN parsers. If you need to indicate a skip move (like in zugzwang analysis), you should do it via comments or a clearly non-PGN annotation.
- **Engine-specific comments:** If you export games analyzed by engines, you’ll often have long comments like `{[%eval 0.53] [%depth 15]}` or multi-line engine analysis in comments. This is fine (PGN will ignore them), but it can bloat files and slow down parsing. Consider an option to strip or compress these if not needed for all users. If writing a parser, you can simply treat them as opaque comments (or parse the `%` tags if you want to use them).
- **Character encoding:** The PGN standard assumes ASCII (or ISO 8859-1) for compatibility ⁵. If you include UTF-8 characters (say, chess piece Unicode symbols, or names with non-Latin scripts), some older tools might not handle them. Modern engines and databases are more tolerant (often assuming UTF-8). As a best practice, output plain ASCII for moves and tags. For player names or comments with accents or non-Latin letters, UTF-8 is usually acceptable today, but if you run into issues, you might fall back to simple transliterations or the original Latin1. Always declare the encoding if possible when sharing PGN (e.g., mention in documentation that your PGN is UTF-8 encoded).
- **Trailing whitespace and line breaks:** Follow the export format guidelines: one move per line or break lines at 80 chars if possible, and ensure there’s a newline after the last move’s result (some

readers expect a newline at end of file). Avoid any trailing spaces as they can cause checksum issues or just look messy.

- **Interoperability of Tags:** Use standard tag names where possible (many programs recognize the same tags). For example, use `ECO` for opening code, not a custom tag like `OpeningCode`. If you add custom tags (you are free to do so), be aware other programs will simply ignore tags they don't recognize. That's fine, but keep names alphanumeric to be safe and document their meaning if you share the PGN.
- **Ensure Result coherence:** This is worth repeating – a surprisingly common error is a mismatch between the result tag and the final marker (e.g., `[Result "1-0"]` but the moves end in `0-1`). This will confuse databases. Always set these correctly in tandem ⁶⁸.
- **Handling of ... in SAN vs. result:** The character `0` vs zero `0` can confuse novices: remember PGN uses zero in results and letter O in castling. So do not mix them. Likewise, don't accidentally use the letter `0` in `1-0` (that should be one-zero). Some font or OCR issues have caused errors here, so just be mindful when generating as text.
- **Multiple games in one file:** If you output many games to one PGN, separate them with an empty line. Most PGN readers handle that fine. If a game is missing a Result tag or termination marker, some readers might try to fuse games or get confused – so even for incomplete games, always include `Result "*"`. If a game is adjourned or not played, you might still include it with `*` result to indicate no result.
- **Testing:** Always test your generated PGN files on a few popular programs (e.g., an open-source library like **pgn4web**, or GUI like **SCID**, or an online validator) to catch any formatting quirks.

By adhering to the PGN standard and the practices above, you can ensure that your PGN files (even those with very complex analysis) remain interoperable across chess software. PGN is quite robust and widespread – a correctly generated PGN with rich annotations allows for sharing deep analysis in a single text file that can be opened and studied by anyone from a simple text editor to advanced chess databases ⁶⁹.

References:

- PGN Specification, 1994 (S.J. Edwards et al.) ⁹ ⁴⁶ ⁷
- Tim Harding, *Portable Game Notation explained* ⁷⁰ ⁴⁷
- Chess StackExchange – discussions on PGN variations and NAG usage ⁴⁸ ⁵⁰
- *PGN Tags and Supplemental Info* ¹² ²⁵ (standard tag roster and optional tags)
- *Numeric Annotation Glyphs list* (PGN spec section 10) ⁵⁰ ⁶³.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 Standard: Portable Game Notation Specification and Implementation Guide
<http://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm>

⁴⁷ ⁴⁸ ⁶⁹ ⁷⁰ Valid PGN variations? - Chess Stack Exchange
<https://chess.stackexchange.com/questions/18214/valid-pgn-variations>